

Part 1

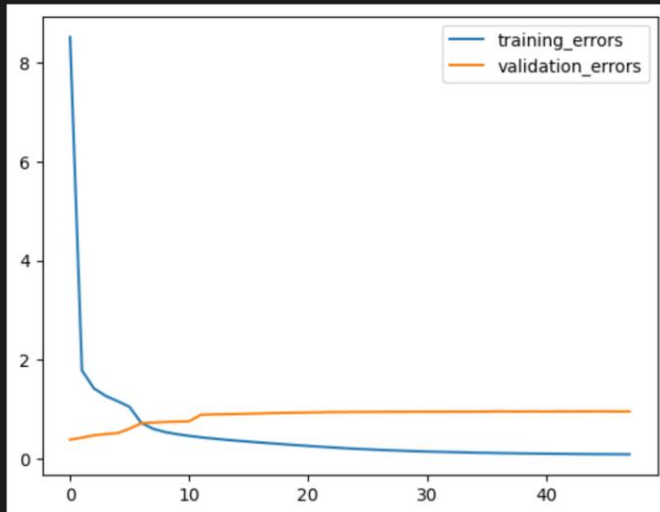
We've designed a model to build and study a neural network with the task of classifying handwritten digits from the MNIST dataset, focusing on numbers 0 to 4. The original architecture consisted of 16 neurons in the first hidden layer, and 32 in the second hidden layer. The model was trained with a learning rate of 0.001, a maximum batch size of 100, and up to 1000 epochs of training. An early stopping rule was defined to avoid overfitting. The initial training process showed that the algorithm was learning since there was a decrease in training loss; however, there were signs of overfitting with the model's training loss being lower than validation loss. The validation score improved continually, but reached a plateau after a certain number of iterations which indicates the model was not learning to generalize to unseen data due to overfitting the training set.

```
Iteration 1, loss = 1.82006439
Validation score: 0.711438
Iteration 2, loss = 0.46993607
Validation score: 0.925817
Iteration 3, loss = 0.29565803
Validation score: 0.945752
Iteration 4, loss = 0.21170881
Validation score: 0.947712
Iteration 5, loss = 0.17604684
Validation score: 0.942157
Iteration 6, loss = 0.15276860
Validation score: 0.955882
Iteration 7, loss = 0.14292986
Validation score: 0.959804
Iteration 8, loss = 0.13927926
Validation score: 0.960458
Iteration 9, loss = 0.13078791
Validation score: 0.958170
Iteration 10, loss = 0.12810735
Validation score: 0.961765
Iteration 11, loss = 0.12019097
Validation score: 0.961111
Iteration 12, loss = 0.11590808
Validation score: 0.964052
Iteration 13, loss = 0.11904348
...
Validation score: 0.967320
Iteration 31, loss = 0.09053425
Validation score: 0.965033
Validation score did not improve more than tol=0.000100 for 10 consecutive epochs. Stopping.
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

MLPClassifier

MLPClassifier(batch_size=100, early_stopping=True, hidden_layer_sizes=[16, 32], max_iter=1000, random_state=42, verbose=True)

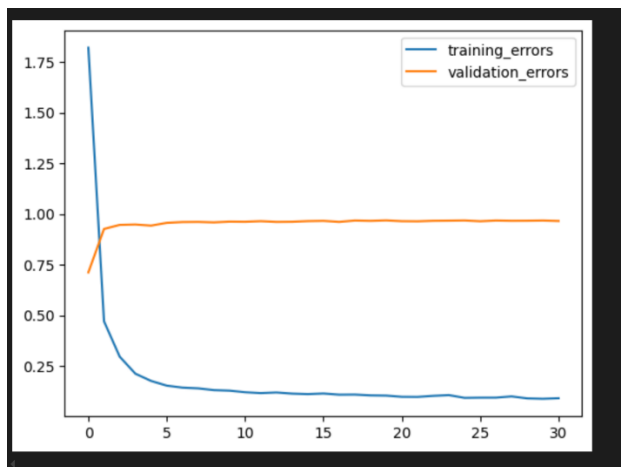
We tested the effect of learning rate on the functionality of the model with learning rates of 1, 0.1, 0.01, 0.001, and 0.0001. Learning rates like one, 0.1, and 0.01 had chaotic training dynamics. The validation scores were highly volatile with many instances of the process getting stopped by the early stopping mechanism. This indicates that at high learning rates, the updates were excessive and instead of achieving convergence, the model kept revolving around the minimum value of the loss function. In contrast, lower learning rates like 0.001 and 0.0001 were stable, but the model still behaved in a way indicative of overfitting. Though this approach was more stable in the control process, plateaued validation scores demonstrated lack of generalizability within the model. While the lower learning rates encouraged gradual learning, they eventually led to overfitting, indicating that additional regularization approaches or modifications to the model's structure were required.



The higher the learning rate the sooner our model overfits.

We then tested influence of batch size on model performance batch sizes of 10, 100, 1000, 500, and 800 while keeping the learning rate constant. Different batch sizes led to variations in the model's ability to generalize. as grid search or random search, to identify optimal parameter combinations.

```
=====learning rate = 1
Iteration 1, loss = 2.22497387
Validation score: 0.203595
Iteration 2, loss = 1.76460397
Validation score: 0.203595
Iteration 3, loss = 1.75250383
Validation score: 0.203595
Iteration 4, loss = 1.75439969
Validation score: 0.211438
Iteration 5, loss = 1.75133551
Validation score: 0.211438
Iteration 6, loss = 1.76585444
Validation score: 0.211438
Iteration 7, loss = 1.75459962
Validation score: 0.195752
Iteration 8, loss = 1.74127834
Validation score: 0.211438
Iteration 9, loss = 1.74580575
Validation score: 0.185948
Iteration 10, loss = 1.73487984
Validation score: 0.203595
Iteration 11, loss = 1.73527048
Validation score: 0.203595
Iteration 12, loss = 1.73350286
Validation score: 0.203595
...
Validation score: 0.959150
Iteration 48, loss = 0.09276780
Validation score: 0.959804
Validation score did not improve more than tol=0.000100 for 10 consecutive epochs. Stopping.
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```



We checked how well the model performed by applying a confusion matrix. The model was correct with different digits to a certain extent as shown in the confusion matrix. The confusion matrix reveals that the neural network model performs with high accuracy for digits 0 and 1, but exhibits more difficulty in correctly classifying digits 2, 3, and 4. While the diagonal values indicate a strong overall performance, the off-diagonal elements highlight specific misclassifications, notably between the visually similar digits 2, 3, and 4. This analysis is crucial for guiding project tasks such as hyperparameter tuning, where the impact of different settings on specific digit classification can be observed, and for error analysis, which can inform strategies like data augmentation or architectural adjustments to improve the model's ability to distinguish between challenging digits. Ultimately, addressing these misclassifications will be key to enhancing the model's generalization and overall effectiveness in the MNIST digit classification task.

```
from sklearn.metrics import confusion_matrix
```

```
test = test[test['label']<5]
```

```
y_test = test['label']
```

```
test = test.drop(['label'], axis=1)
```

```
y_pred = nn.predict(test)
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
print(cm)
```

```
[[ 967   1   6   5   1]
 [   0 1117  10   5   3]
 [  11   7  947  48  19]
 [   6   4  115 878   7]
 [   4   6   18   1 953]]
```

Part 2

We made a Naïve Bayes classifier to classify news headlines as either "real" or "fake". To prepare the data, we converted each headline into a numerical feature vector, which allowed us to represent the frequency of each word in the headline. We split the dataset into 70% training and 30% test sets, using a random state of 5 to ensure reproducibility. To improve the model's performance, we filtered out words that appeared in more than 70% or less than 0.5% of the headlines, as these words were considered uninformative for classification.

For each test sample, we calculated the posterior probabilities based on the word frequencies from the training set. By comparing the log probabilities of the two classes, we predicted whether a headline was real or fake. Our model achieved an accuracy of 81.12% on the test set. To better understand the model's performance, we analysed the confusion matrix. We found that 300 fake headlines were correctly classified as fake (True Negatives), and 495 real headlines were correctly classified as real (True Positives). However, the model misclassified 90 fake headlines as real (False Positives) and 95 real headlines as fake (False Negatives).

Output for part 2: (random state 5)

Accuracy: 0.8112

Confusion Matrix:

```
[[300  90]
 [ 95 495]]
```

Confusion matrix

	Predicted fake == 1	Predicted real == 1
Actual fake == 1	True negative	False positive
Actual real == 1	False negative	True positive

```
Accuracy: 0.8112
```

```
Confusion Matrix:
```

```
[[300  90]
 [ 95 495]]
```